DURAND Antonin
MASSINOND Matéo
MENON Valentin
RENVOISÉ Yann

Ingé2 LSI2 Groupe2 - 2025/2026

Goat Pharmacy

Conception et implémentation d'une base de données répartie pour la gestion d'une chaîne de pharmacies / cliniques sur deux villes

Introduction 3
A. Présentation du besoin 4
B. Schéma global 5
C. Stratégie de fragmentation 6
1. Principe général de fragmentation 6
2. Fragmentations horizontale 6
2.1. Fragmentation horizontale primaire par ville 6
2.2. Fragmentation horizontale dérivée à partir des fragments parents 7
3. Fragmentation verticale 9
3.1. Fragmentation verticale de la table Patient 9
3.2. Fragmentation verticale de la table Médicament 9
4. Pertinence globale de la stratégie 10
D. Stratégie d'allocation description de l'implémentation PostgreSQL 11
1. Installation et configuration 11
1.1. Initialisation des bases de données 11
1.2. Modification du fichier de configuration 11
1.3. Démarrer les instances PostgreSQL 11
1.4. Connexion aux instances PostgreSQL pour créer un utilisateur et les bases de données 12
2. Implémentation PostgreSQL 12
2.2. Importation du schéma dans les serveurs 15
2.3. Création des fragments 16
3. Gestion des insertions 19
3.1. Serveur Casablanca 19
3.2. Serveur Rabat 21
E. Jeux de données 24
F. Requêtes réparties 29
1. Création des Vues gloables 29
1.1. Vue Patient_Global 29
1.2. Vue Medecin_Global 30
1.3. Vue Consultation_Global 30
1.4. Vue Prescription_Global 31

1.5. Vue LignePrescription_Global..... 31
1.6. Vue Stock_Global..... 32
1.7. Vue Vente_Global..... 32
1.8. Vue LigneVente_Global..... 33
1.9. Vue Medicament_Global..... 33
2. Requêtes réparties sur les vues globales..... 34
2.1. Afficher les patients appartenant au site de Casablanca..... 34
2.2. Afficher les médicaments dont le prix unitaire est maximal..... 34
2.3. Afficher les consultations d'un patient donné avec le nom du médecin..... 35
2.4. Afficher tous les médicaments prescrits lors d'une consultation donnée..... 36
2.5. Afficher le chiffre d'affaires total par ville..... 37
2.6. Afficher le médicament le plus vendu sur l'ensemble du réseau..... 37
2.7. Afficher les médicaments dont le stock est inférieur au seuil d'alerte..... 38
2.8. Afficher, pour chaque médecin, le nombre total de consultations réalisées..... 39
2.9. Afficher les patients ayant effectué une consultation dans une ville et un achat dans l'autre..... 40
2.10. Afficher les patients ayant acheté au moins un médicament qui figurait dans leur prescription 41
G. Analyse critique..... 43
H. Conclusion..... 44

Introduction

Dans le cadre du cours de bases de données réparties, nous devons réaliser un projet en équipe qui consiste à concevoir, implémenter puis démontrer l'utilisation d'une base de données multi-sites. Ce projet va permettre de comprendre comment en pratique séparer des données sur plusieurs serveurs. Cette répartition nous permettra également de constater des gains de performance, d'autonomie locale, de réduction de trafic réseau et de disponibilité logique. L'architecture conçue devrait être transparente pour les utilisateurs.

Au lieu de centraliser toutes les informations sur un seul et même serveur, une architecture répartie consiste au contraire à les distribuer sur plusieurs machines. Cette approche nous permet plusieurs choses telles que : améliorer les performances, limiter les échanges réseau inutiles et augmenter la disponibilité du système en cas de problèmes sur les serveurs. Pour arriver à un tel système, nous devons nous appuyer sur l'utilisation de mécanismes de fragmentation, consistant à répartir les données selon certains critères. Nous nous servirons également de la réplication de données pour rendre certaines informations particulières accessibles sur plusieurs sites.

Un des facettes importantes de ce type d'architecture complexe est de garder une certaine transparence de localisation. L'idée est que l'utilisateur doit être en mesure d'interroger la base directement comme si elle était unique, sans avoir besoin de se soucier de l'endroit où les données sont réellement stockées. Cela implique donc de préparer des vues globales à partir de fragments répartis, ce qui demande une bonne organisation des données et des accès.

La gestion des transactions et des accès concurrents est également un aspect important. Même si les données sont distribuées, il faut garantir leur cohérence à tout moment et éviter ainsi les conflits lors des opérations en simultané. Enfin, l'optimisation des requêtes est un des objectifs principaux, ce qui nous permettrait d'exécuter un maximum de traitements localement afin de réduire ainsi les coûts de transfert d'informations entre les différents sites.

Dans ce rapport, nous allons présenter les choix de conceptions de la base et de sa fragmentation, la mise en place technique de la base répartie, ainsi que les vues globales et les requêtes permettant d'exploiter les données à l'échelle du système.

Toutes les données utilisées lors de ce projet sont accessibles dans le repository publique suivant : https://github.com/DurandAntonin/GoatPharmacy

Lien vers la présentation orale :
https://docs.google.com/presentation/d/1q6FRz0QCyLDFFL1n0sWgZytd5rihOp2AWJaWbDAF9aI/

A. Présentation du besoin

Dans notre contexte, l'entreprise de santé privée est répartie sur deux villes.
Si on met en place une architecture centralisée, chaque requête va parcourir les données des deux villes alors qu'on n'en a pas besoin dans la plupart des cas. Il est par exemple inutile de faire une requête qui parcourt les stocks de Rabat pour chercher ceux de Casablanca.

Une architecture répartie est donc pertinente pour séparer les données entre celles de Casablanca et celles de Rabat. Ainsi, si une requête ne concerne qu'une seule des deux villes, elle ne parcourra que les données la concernant. On veut séparer les informations au niveau des lignes des tables qui contiennent l'attribut "Ville". Nous utilisons donc la fragmentation horizontale pour ce projet.

Pour optimiser davantage l'accès aux données, nous appliquons également une fragmentation verticale sur la table Patient où nous séparons les informations d'identification nécessaires aux besoins administratifs (Nom, Prénom, Naissance) des informations privées ou de contact (Téléphone, Adresse) et sur la table Médicament où nous séparons les informations techniques (Forme, Dosage, Fabricant) du prix, ce qui permet d'avoir une gestion indépendante des tarifs en fonction du site.

Dans chacune des deux villes, on peut stocker les informations des patients, des médecins, des ventes et des stocks puisqu'ils dépendent tous de l'attribut "Ville".
Ensuite, on sépare également pour chaque ville les informations concernant les consultations, les prescriptions ainsi que les tables "LignePrescription" et "LigneVente". Ces tables n'ont pas de colonne "Ville" mais elles dépendent des tables qui en ont une.
Par contre, la table "Médicament" doit être présente sur les deux sites puisqu'elle ne dépend pas de la ville. Cela permet d'éviter de trop nombreuses requêtes réseau.

Grâce à ce système, on attend de nombreux gains en terme de :

- performance : les requêtes peuvent être exécutées simultanément sur les deux serveurs, la charge est répartie sur deux processeurs au lieu d'un seul. De plus, en fragmentant les tables, chaque serveur manipule des index plus petits. Les recherches dans la base de données sont donc plus rapides.
- d'autonomie locale : le personnel de Casablanca gère ses propres patients et stocks sans dépendre du serveur de Rabat et inversement. Chaque ville garde le contrôle de ses données. En cas de maintenance sur le serveur de Casablanca , les activités médicales de Rabat ne sont pas interrompues.
- réduction du trafic réseau : la plupart des requêtes concernent des données locales à la ville. En stockant les données là où elles sont utilisées, on évite de requêter des serveurs lointains.
- disponibilité logique : en cas de panne d'une des deux villes, le système est toujours fonctionnel pour l'autre ville (à part peut-être pour les médicaments).

B. Schéma global

https://drawsql.app/teams/antonind/diagrams/goatpharmacy-schema/embed

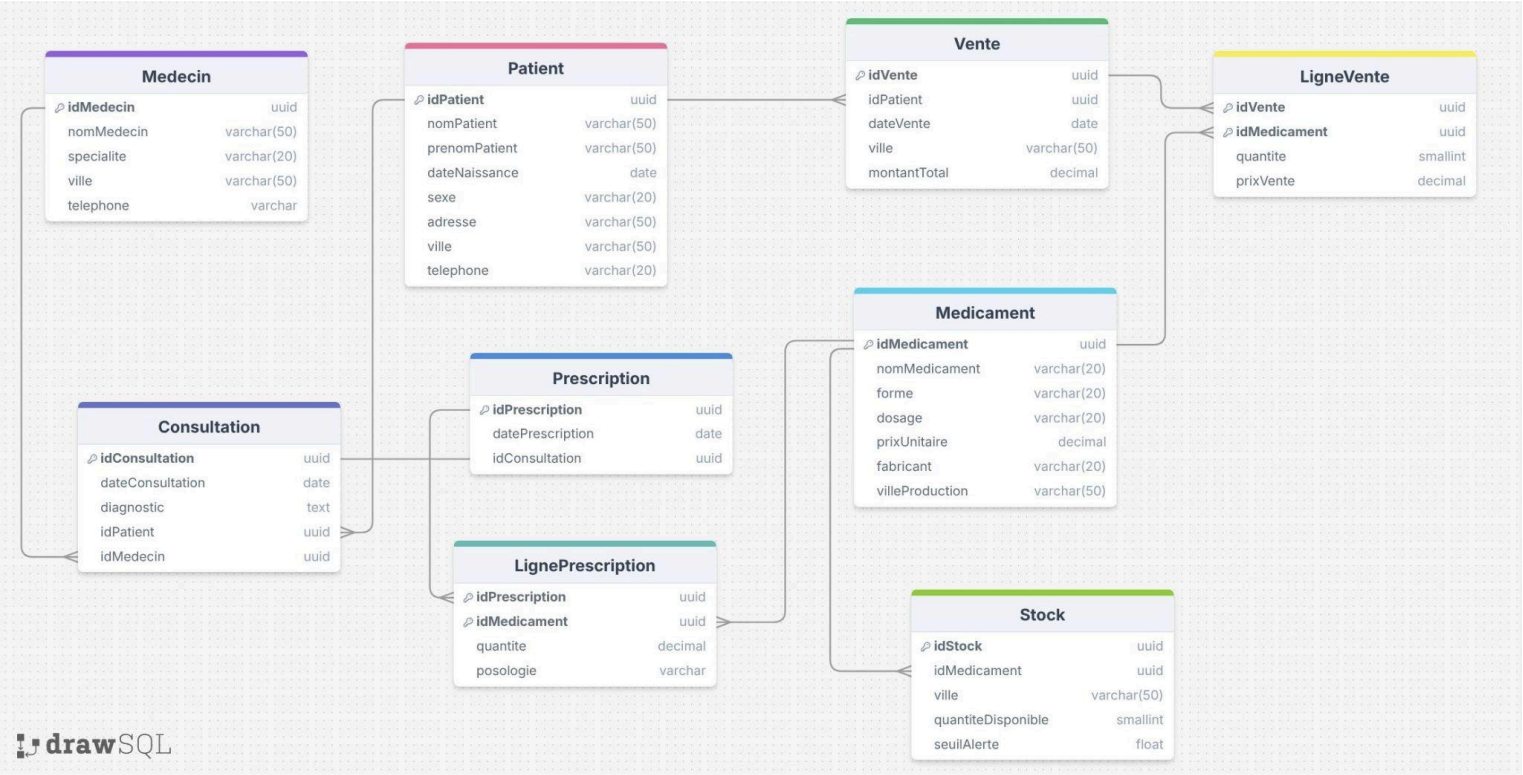

Schéma 1 - Schéma de la base de données globale

C. Stratégie de fragmentation

1. Principe général de fragmentation

Dans le cadre de cette base de données répartie entre les deux sites S1 (Casablanca) et S2 (Rabat), la fragmentation des données à plusieurs objectifs :

- rapprocher les données des utilisateurs locaux
- réduire les accès distants
- améliorer les performances
- garantir l'autonomie de chaque site.

Comme dit précédemment, nous avons décidé d'adopter une approche de fragmentation mixte combinant :

- fragmentation horizontale (par ville) ;
- fragmentation verticale (par type d'usage).

Dans chaque type de fragmentation, une donnée commune telle que l'Id du sujet en question doit être dupliquée sur chaque fragmentation, afin de rendre la jointure et donc le regroupement des informations possible.

2. Fragmentations horizontale

Définition Fragmentation Horizontale (Séparation par lignes) : La relation est fragmentée en groupes de tuples de sorte que chaque tuple est attribué à au moins un fragment. Pour les tables suivantes nous allons utiliser ici la ville comme critère de fragmentation.

2.1. Fragmentation horizontale primaire par ville

- Médecins

Les médecins exercent principalement dans la clinique de leur ville.

Fragmentation

- **Medecin_S1** → médecins où Ville = 'Casablanca'
- **Medecin_S2** → médecins où Ville = 'Rabat'

Justification

- Les médecins sont rattachés à une structure locale ;
- Leurs consultations sont réalisées majoritairement dans leur propre ville ;
- Cela permet de garder localement les données nécessaires à l'activité médicale quotidienne ;
- Les jointures avec les consultations sont plus rapides lorsqu'elles restent sur le même site.

- Patients

Les patients sont principalement gérés dans leur ville d'enregistrement.

Fragmentation :

- **Patient_S1 (Casablanca)** → patients où Ville = 'Casablanca'
- **Patient_S2 (Rabat)** → patients où Ville = 'Rabat'

Justification :

- Les consultations et ventes sont locales dans la majorité des cas ;
- Réduction des accès réseau ;
- Meilleure performance pour les requêtes locales.

- Stock

Chaque ville possède son propre stock de médicaments.

Fragmentation

- **Stock_S1** → stock où Ville = 'Casablanca'
- **Stock_S2** → stock où Ville = 'Rabat'

Justification

- Gestion totalement locale des stocks ;
- Mise à jour fréquente → doit rester locale ;
- Réduction du trafic réseau.

- Ventes

Les ventes sont réalisées dans la pharmacie locale.

Fragmentation

- **Vente_S1** → ventes où Ville = 'Casablanca'
- **Vente_S2** → ventes où Ville = 'Rabat'

Justification

- Les ventes sont propres à chaque site ;
- Accès rapide pour la gestion commerciale locale ;
- Agrégation globale possible via vues.

2.2. Fragmentation horizontale dérivée à partir des fragments parents

- Consultations

Les consultations ne possèdent pas directement l'attribut "Ville" mais sont réalisées dans une clinique locale elles-mêmes rattachées à un site via le médecin.

Fragmentation

- **Consultation_S1** → consultations liées à des médecins de Casablanca
- **Consultation_S2** → consultations liées à des médecins de Rabat

Justification

- Les consultations sont strictement locales ;
- Accès rapide pour les médecins ;
- Pas de dépendance inter-site fréquente.

- Prescriptions

Les prescriptions ne possèdent pas directement l'attribut "Ville", mais elles dépendent des consultations, elles-mêmes rattachées à un site via le médecin.

Fragmentation

- **Prescription_S1** → prescriptions liées aux consultations de Casablanca
- **Prescription_S2** → prescriptions liées aux consultations de Rabat

Justification

- Les prescriptions suivent naturellement la localisation des consultations ;
- Cela évite de séparer des données fortement liées entre plusieurs sites ;
- Les accès aux prescriptions restent locaux pour le personnel médical de chaque ville ;
- Cette fragmentation améliore les performances et réduit les échanges réseau lors des traitements courants.

- Ligne_Prescription

Les lignes de prescription ne possèdent pas directement l'attribut "Ville", mais elles dépendent des prescriptions, elles-mêmes rattachées à une consultation localisée.

Fragmentation

- **LignePrescription_S1** → lignes de prescription liées aux prescriptions de Casablanca
- **LignePrescription_S2** → lignes de prescription liées aux prescriptions de Rabat

Justification

- Les lignes de prescription sont directement liées aux prescriptions, elles-mêmes locales à une ville ;
- Cela permet de conserver la cohérence des données médicales sur un même site ;
- Les requêtes liées aux prescriptions (affichage, suivi, analyse) sont exécutées localement ;
- La fragmentation réduit les accès inter-sites et améliore les performances globales.

- Ligne_Vente

Les lignes de vente ne possèdent pas directement l'attribut "Ville", mais elles dépendent des ventes, qui sont localisées par ville.

Fragmentation

- **LigneVente_S1** → lignes de vente liées aux ventes de Casablanca
- **LigneVente_S2** → lignes de vente liées aux ventes de Rabat

Justification

- Les lignes de vente sont associées aux ventes locales réalisées dans chaque pharmacie ;
- Cela garantit que toutes les informations relatives à une vente restent sur le même site ;
- Les opérations commerciales (facturation, analyse des ventes) peuvent être effectuées localement ;
- La fragmentation permet de limiter le trafic réseau et d'améliorer la rapidité des requêtes.

- **Medicament_Prix**

La table Medicament fragmentée verticalement (voir plus bas) ne possède pas directement l'attribut "Ville", mais on peut émettre l'hypothèse que les prix d'un même médicament peuvent être différents d'une ville à l'autre.

Fragmentation

- **Medicament_Prix_S1** → prix des médicaments à Casablanca
- **Medicament_Prix_S2** → prix des médicaments à Rabat

Justification

- Les prescriptions suivent naturellement la localisation des consultations ;
- Cela évite de séparer des données fortement liées entre plusieurs sites ;
- Les accès aux prescriptions restent locaux pour le personnel médical de chaque ville ;
- Cette fragmentation améliore les performances et réduit les échanges réseau lors des traitements courants.

3. Fragmentation verticale

Définition Fragmentation Verticale (Séparation par colonnes) : Le schéma de la relation est divisé en schémas plus petits. Chaque fragment doit contenir une clé candidate commune afin d'assurer une liaison sans perte.

3.1. Fragmentation verticale de la table Patient

Nous voulons séparer les informations d'identifications nécessaires pour les besoins administratifs, des informations privées moins utiles telles que les informations de contact.

Fragmentation

- **Patient_Info**
  - IdPatient
  - NomPatient
  - PrenomPatient
  - DateNaissance
  - Sexe
- **Patient_Contact**
  - IdPatient
  - Telephone
  - Adresse
  - Ville

Justification

- Sécurité des données (données sensibles isolées) ;
- Optimisation des requêtes (certaines n'utilisent que les infos d'identité) ;
- Réduction du volume de données manipulées.
- Possibilité de faire un annuaire de contact sans données additionnelles.

3.2. Fragmentation verticale de la table Médicament

Nous voulons pouvoir séparer les informations commerciales, tel que le prix qui est “volatile”, des informations techniques données par le fabricant qui ne changent pas ou peu.

Fragmentation

- **Medicament_Info**
  - IdMedicament
  - NomMedicament
  - Forme
  - Dosage
  - Fabricant
  - VilleProduction
- **Medicament_Prix**
  - IdMedicament
  - PrixUnitaire

Justification

- Les prix changent plus fréquemment → gestion indépendante ;
- Optimisation des requêtes métier (ex : stock sans prix) ;
- Meilleure modularité.

4. Pertinence globale de la stratégie

Cette stratégie est adaptée aux besoins fonctionnels décrits dans le projet :

- Les données opérationnelles (consultations, ventes, stock) sont localisées par ville ;
- Les données globales (patients, médicaments) restent reconstructibles via des vues globales ;
- Le système garantit :
  a. **performance locale élevée** ;
  b. **réduction du trafic réseau** ;
  c. **autonomie des sites** ;
  d. **transparence pour l’utilisateur final**.

Pour la table Médicament, les données représentent des types de médicaments précis, qui sont généraux et ne dépendent pas de la ville. En supposant que les prix sont les mêmes entre Casablanca et Rabat, on pourra dupliquer les données directement lors de l’ajout de la liste de médicament.

D. Stratégie d'allocation description de l'implémentation PostgreSQL

1. Installation et configuration

1.1. Initialisation des bases de données

Tout d'abord, on crée un dossier pour chaque site (Casablanca, Rabat et un S3 pour répliquer la table Médicament). Ensuite, en allant dans le répertoire "C:\Program Files\PostgreSQL\18\bin", on créer un serveur de bases de données postgres pour chaque site :

```shell
.\initdb -D C:\PostgreSQL\data\Casablanca

.\initdb -D C:\PostgreSQL\data\Rabat

.\initdb -D C:\PostgreSQL\data\S3
```

1.2. Modification du fichier de configuration

Il faut modifier le fichier de configuration "postgresql.conf" de chaque serveur pour qu'ils écoutent sur des ports différents. La ligne à modifier est la suivante :

```none
#port = 5432 # (change requires restart)
```

- La DB de Casablanca va écouter sur le port 55001
- La DB de Rabat va écouter sur le port 55002
- La DB de S3 va écouter sur le port 55003

1.3. Démarrer les instances PostgreSQL

On peut démarrer les trois serveurs à l'aide de ces commandes :

```shell
.\postgres.exe -D C:\PostgreSQL\data\Casablanca
.\postgres.exe -D C:\PostgreSQL\data\Rabat
```

```
.\postgres.exe -D C:\PostgreSQL\data\S3
```

1.4. Connexion aux instances PostgreSQL pour créer un utilisateur et les bases de données

Il faut maintenant créer un utilisateur avec les accès administrateurs dans chacun des sites. Il faut aussi créer les base de données correspondantes :

```Shell
.\psql.exe -h localhost -p 55001 postgres
CREATE ROLE postgres WITH LOGIN SUPERUSER PASSWORD 'admin';
CREATE DATABASE db_casablanca;
.\psql.exe -h localhost -p 55002 postgres
CREATE ROLE postgres WITH LOGIN SUPERUSER PASSWORD 'admin';
CREATE DATABASE db_rabat;
.\psql.exe -h localhost -p 55003 postgres
CREATE ROLE postgres WITH LOGIN SUPERUSER PASSWORD 'admin';
CREATE DATABASE db_s3;
```

2. Implémentation PostgreSQL

2.1. Création des tables

Les tables sont créées sur S3 pour pouvoir ensuite effectuer les fragmentations horizontales et verticales sur S1 et S2.

```SQL
CREATE TABLE "Patient"(
    "idPatient" UUID PRIMARY KEY,
    "nomPatient" VARCHAR(50) NOT NULL,
    "prenomPatient" VARCHAR(50) NOT NULL,
    "dateNaissance" DATE NOT NULL,
    "sexe" VARCHAR(20) NOT NULL,
    "adresse" VARCHAR(50) NOT NULL,
    "ville" VARCHAR(50) NOT NULL,
    "telephone" VARCHAR(20) NOT NULL
);

CREATE TABLE "Medecin"(
    "idMedecin" UUID PRIMARY KEY,
```

```sql
    "nomMedecin" VARCHAR(50) NOT NULL,
    "specialite" VARCHAR(20) NOT NULL,
    "ville" VARCHAR(50) NOT NULL,
    "telephone" VARCHAR(255) NOT NULL
);

CREATE TABLE "Consultation"(
    "idConsultation" UUID PRIMARY KEY,
    "dateConsultation" DATE NOT NULL,
    "diagnostic" TEXT NOT NULL,
    "idPatient" UUID NOT NULL,
    "idMedecin" UUID NOT NULL
);

CREATE TABLE "Prescription"(
    "idPrescription" UUID PRIMARY KEY,
    "datePrescription" DATE NOT NULL,
    "idConsultation" UUID NOT NULL
);

CREATE TABLE "LignePrescription"(
    "idPrescription" UUID NOT NULL,
    "idMedicament" UUID NOT NULL,
    "quantite" DECIMAL(8, 2) NOT NULL,
    "posologie" VARCHAR(255) NOT NULL
);

CREATE TABLE "Medicament"(
    "idMedicament" UUID PRIMARY KEY,
    "nomMedicament" VARCHAR(20) NOT NULL,
    "forme" VARCHAR(20) NOT NULL,
    "dosage" VARCHAR(20) NOT NULL,
    "prixUnitaire" DECIMAL(8, 2) NOT NULL,
    "fabricant" VARCHAR(20) NOT NULL,
    "villeProduction" VARCHAR(50) NOT NULL
);

CREATE TABLE "Stock"(
    "idStock" UUID PRIMARY KEY,
    "idMedicament" UUID NOT NULL,
    "ville" VARCHAR(50) NOT NULL,
    "quantiteDisponible" SMALLINT NOT NULL,
    "seuilAlerte" FLOAT(53) NOT NULL
);
```

```sql
CREATE TABLE "Vente"(
    "idVente" UUID PRIMARY KEY,
    "idPatient" UUID NOT NULL,
    "dateVente" DATE NOT NULL,
    "ville" VARCHAR(50) NOT NULL,
    "montantTotal" DECIMAL(8, 2) NOT NULL
);

CREATE TABLE "LigneVente"(
    "idVente" UUID NOT NULL,
    "idMedicament" UUID NOT NULL,
    "quantite" SMALLINT NOT NULL,
    "prixVente" DECIMAL(8, 2) NOT NULL
);

ALTER TABLE
    "LigneVente" ADD PRIMARY KEY("idVente", "idMedicament");
ALTER TABLE
    "LignePrescription" ADD PRIMARY KEY("idPrescription", "idMedicament");

ALTER TABLE
    "Consultation" ADD CONSTRAINT "consultation_idpatient_foreign" FOREIGN
KEY("idPatient") REFERENCES "Patient"("idPatient");
ALTER TABLE
    "LignePrescription" ADD CONSTRAINT
"ligneprescription_idmedicament_foreign" FOREIGN KEY("idMedicament")
REFERENCES "Medicament"("idMedicament");
ALTER TABLE
    "LignePrescription" ADD CONSTRAINT
"ligneprescription_idprescription_foreign" FOREIGN KEY("idPrescription")
REFERENCES "Prescription"("idPrescription");
ALTER TABLE
    "Prescription" ADD CONSTRAINT "prescription_idconsultation_foreign"
FOREIGN KEY("idConsultation") REFERENCES "Consultation"("idConsultation");
ALTER TABLE
    "Stock" ADD CONSTRAINT "stock_idmedicament_foreign" FOREIGN
KEY("idMedicament") REFERENCES "Medicament"("idMedicament");
ALTER TABLE
    "Vente" ADD CONSTRAINT "vente_idpatient_foreign" FOREIGN
KEY("idPatient") REFERENCES "Patient"("idPatient");
ALTER TABLE
    "LigneVente" ADD CONSTRAINT "lignevente_idvente_foreign" FOREIGN
KEY("idVente") REFERENCES "Vente"("idVente");
```

```
ALTER TABLE
    "LigneVente" ADD CONSTRAINT "lignevente_idmedicament_foreign" FOREIGN
KEY("idMedicament") REFERENCES "Medicament"("idMedicament");
ALTER TABLE
    "Consultation" ADD CONSTRAINT "consultation_idmedecin_foreign" FOREIGN
KEY("idMedecin") REFERENCES "Medecin"("idMedecin");
```

2.2. Importation du schéma dans les serveurs

Sur “db_casablanca” :

```sql
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

CREATE SERVER serveur_rabat FOREIGN DATA WRAPPER postgres_fdw OPTIONS
(host 'localhost', port '55002', dbname 'db_rabat');

CREATE SERVER serveur_s3 FOREIGN DATA WRAPPER postgres_fdw OPTIONS (host
'localhost', port '55003', dbname 'db_s3');

CREATE USER MAPPING FOR postgres SERVER serveur_rabat OPTIONS (user
'postgres', password 'admin'); CREATE USER MAPPING FOR postgres SERVER
serveur_s3 OPTIONS (user 'postgres', password 'admin');

DROP SCHEMA IF EXISTS rabat_schema CASCADE;
CREATE SCHEMA rabat_schema;
IMPORT FOREIGN SCHEMA public
FROM SERVER serveur_rabat INTO rabat_schema;

DROP SCHEMA IF EXISTS s3_schema CASCADE;
CREATE SCHEMA s3_schema;
IMPORT FOREIGN SCHEMA public
 FROM SERVER serveur_s3 INTO s3_schema;
```

Sur “db_rabat” :

```sql
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

CREATE SERVER serveur_casa FOREIGN DATA WRAPPER postgres_fdw OPTIONS (host
'localhost', port '55001', dbname 'db_casablanca');
```

```sql
CREATE SERVER serveur_s3 FOREIGN DATA WRAPPER postgres_fdw OPTIONS (host
'localhost', port '55003', dbname 'db_s3');

CREATE USER MAPPING FOR postgres SERVER serveur_casa OPTIONS (user
'postgres', password 'admin'); CREATE USER MAPPING FOR postgres SERVER
serveur_s3 OPTIONS (user 'postgres', password 'admin');

DROP SCHEMA IF EXISTS casa_schema CASCADE;
CREATE SCHEMA casa_schema;
IMPORT FOREIGN SCHEMA public
FROM SERVER serveur_casa
INTO casa_schema;

DROP SCHEMA IF EXISTS s3_schema CASCADE;
CREATE SCHEMA s3_schema;
IMPORT FOREIGN SCHEMA public
FROM SERVER serveur_s3 INTO s3_schema;
```

2.3. Création des fragments

2.3.1. Serveur Casablanca

```sql
DROP TABLE IF EXISTS Patient_S1 CASCADE;
DROP TABLE IF EXISTS Patient_Info_S1 CASCADE;
DROP TABLE IF EXISTS Patient_Contact_S1 CASCADE;
DROP TABLE IF EXISTS Medecin_S1 CASCADE;
DROP TABLE IF EXISTS Consultation_S1 CASCADE;
DROP TABLE IF EXISTS Prescription_S1 CASCADE;
DROP TABLE IF EXISTS LignePrescription_S1 CASCADE;
DROP TABLE IF EXISTS Medicament_Prix_S1 CASCADE;
DROP TABLE IF EXISTS Stock_S1 CASCADE;
DROP TABLE IF EXISTS Vente_S1 CASCADE;
DROP TABLE IF EXISTS LigneVente_S1 CASCADE;

CREATE TABLE Patient_S1
AS SELECT * FROM s3_schema."Patient"
WHERE ville = 'Casablanca';
```

```sql
CREATE TABLE Patient_Info_S1
AS SELECT "idPatient", "nomPatient", "prenomPatient", "dateNaissance",sexe
FROM Patient_S1;

CREATE TABLE Patient_Contact_S1
AS SELECT "idPatient",telephone, adresse, ville
FROM Patient_S1;

CREATE TABLE Medecin_S1
AS SELECT * FROM s3_schema."Medecin"
WHERE ville = 'Casablanca';

CREATE TABLE Consultation_S1
AS SELECT * FROM s3_schema."Consultation"
WHERE "idMedecin" IN (SELECT "idMedecin" FROM Medecin_S1);

CREATE TABLE Prescription_S1
AS SELECT * FROM s3_schema."Prescription"
WHERE "idConsultation" IN (SELECT "idConsultation" FROM Consultation_S1);

CREATE TABLE LignePrescription_S1
AS SELECT * FROM s3_schema."LignePrescription"
WHERE "idPrescription" IN (SELECT "idPrescription" FROM Prescription_S1);

CREATE TABLE Medicament_Prix_S1
AS SELECT "idMedicament", "prixUnitaire"
FROM s3_schema."Medicament";

CREATE TABLE Stock_S1
AS SELECT * FROM s3_schema."Stock"
WHERE ville = 'Casablanca';

CREATE TABLE Vente_S1
AS SELECT * FROM s3_schema."Vente"
WHERE ville = 'Casablanca';

CREATE TABLE LigneVente_S1
AS SELECT * FROM s3_schema."LigneVente"
WHERE "idVente" in (SELECT "idVente" FROM Vente_S1);
```

2.3.2. Serveur Rabat

```sql
DROP TABLE IF EXISTS Patient_S2 CASCADE;
DROP TABLE IF EXISTS Patient_Info_S2 CASCADE;
DROP TABLE IF EXISTS Patient_Contact_S2 CASCADE;
DROP TABLE IF EXISTS Medecin_S2 CASCADE;
DROP TABLE IF EXISTS Consultation_S2 CASCADE;
DROP TABLE IF EXISTS Prescription_S2 CASCADE;
DROP TABLE IF EXISTS LignePrescription_S2 CASCADE;
DROP TABLE IF EXISTS Medicament_Prix_S2 CASCADE;
DROP TABLE IF EXISTS Stock_S2 CASCADE;
DROP TABLE IF EXISTS Vente_S2 CASCADE;
DROP TABLE IF EXISTS LigneVente_S2 CASCADE;

CREATE TABLE Patient_S2
AS SELECT * FROM s3_schema."Patient"
WHERE ville = 'Rabat';

CREATE TABLE Patient_Info_S2
AS SELECT "idPatient", "nomPatient", "prenomPatient", "dateNaissance",
"sexe"
FROM Patient_S2;

CREATE TABLE Patient_Contact_S2
AS SELECT "idPatient","telephone", "adresse", "ville"
FROM Patient_S2;

CREATE TABLE Medecin_S2
AS SELECT * FROM s3_schema."Medecin"
WHERE ville = 'Rabat';

CREATE TABLE Consultation_S2
AS SELECT * FROM s3_schema."Consultation"
WHERE "idMedecin" IN (SELECT "idMedecin" FROM Medecin_S2);

CREATE TABLE Prescription_S2
AS SELECT * FROM s3_schema."Prescription"
WHERE "idConsultation" IN (SELECT "idConsultation" FROM Consultation_S2);

CREATE TABLE LignePrescription_S2
AS SELECT * FROM s3_schema."LignePrescription"
WHERE "idPrescription" IN (SELECT "idPrescription" FROM Prescription_S2);

CREATE TABLE Medicament_Prix_S2
AS SELECT "idMedicament", "prixUnitaire"
FROM s3_schema."Medicament";
```

```sql
CREATE TABLE Stock_S2
AS SELECT * FROM s3_schema."Stock"
WHERE ville = 'Rabat';

CREATE TABLE Vente_S2
AS SELECT * FROM s3_schema."Vente"
WHERE ville = 'Rabat';

CREATE TABLE LigneVente_S2
AS SELECT * FROM s3_schema."LigneVente"
WHERE "idVente" IN (SELECT "idVente" FROM Vente_S2);
```

3. Gestion des insertions

On va créer des triggers pour gérer la cohérence des données entre les serveurs S1 et S2

3.1. Serveur Casablanca

```sql
DROP TRIGGER IF EXISTS trigger_patient_s1 ON s3_schema."Patient";
DROP TRIGGER IF EXISTS trigger_medecin_s1 ON Medecin_S1;
DROP TRIGGER IF EXISTS trigger_medicament_s1 ON s3_schema."Medicament";
DROP TRIGGER IF EXISTS trigger_stock_s1 ON Stock_S1;
DROP TRIGGER IF EXISTS trigger_vente_s1 ON Vente_S1;

-- Patient
CREATE OR REPLACE FUNCTION process_patient_s1()
RETURNS TRIGGER AS $$BEGIN
    IF NEW.ville = 'Casablanca' THEN
        INSERT INTO Patient_Info_S1 ("idPatient", "nomPatient",
"prenomPatient", "dateNaissance", sexe)
        VALUES (NEW."idPatient", NEW."nomPatient", NEW."prenomPatient",
NEW."dateNaissance", NEW."sexe");

        INSERT INTO Patient_Contact_S1 ("idPatient", telephone, adresse,
ville)
        VALUES (NEW."idPatient", NEW."telephone", NEW."adresse",
NEW."ville");
    ELSIF NEW.ville = 'Rabat' THEN
```

```
        INSERT INTO rabat_schema.Patient_Info_S2 ("idPatient",
"nomPatient", "prenomPatient", "dateNaissance", sexe)
        VALUES (NEW."idPatient", NEW."nomPatient", NEW."prenomPatient",
NEW."dateNaissance", NEW."sexe");

        INSERT INTO rabat_schema.Patient_Contact_S2 ("idPatient",
"telephone", "adresse", "ville")
        VALUES (NEW."idPatient", NEW."telephone", NEW."adresse",
NEW."ville");
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER trigger_patient_s1
BEFORE INSERT ON s3_schema."Patient"
FOR EACH ROW
EXECUTE FUNCTION process_patient_s1();

-- Medecin
CREATE OR REPLACE FUNCTION process_medecin_s1()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.ville = 'Casablanca' THEN
        RETURN NEW;
    ELSIF NEW.ville = 'Rabat' THEN
        INSERT INTO rabat_schema.Medecin_S2 VALUES (NEW.*);
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER trigger_medecin_s1
BEFORE INSERT
ON Medecin_S1
FOR EACH ROW EXECUTE FUNCTION process_medecin_s1();

-- Medicament
CREATE OR REPLACE FUNCTION process_medicament_s1()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Medicament_Prix_S1 ("idMedicament", "prixUnitaire")
    VALUES (NEW."idMedicament", NEW."prixUnitaire");
    RETURN NULL;
```

```
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER trigger_medicament_s1
BEFORE INSERT
ON s3_schema."Medicament"
FOR EACH ROW EXECUTE FUNCTION process_medicament_s1();

-- Stock
CREATE OR REPLACE FUNCTION process_stock_s1()
RETURNS TRIGGER AS $$BEGIN
    IF NEW.ville = 'Casablanca' THEN
        RETURN NEW;
    ELSIF NEW.ville = 'Rabat' THEN
        INSERT INTO rabat_schema.Stock_S2 VALUES (NEW.*);
    END IF;
    RETURN NULL;
END;$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER trigger_stock_s1
BEFORE INSERT ON Stock_S1
FOR EACH ROW EXECUTE FUNCTION process_stock_s1();

-- Vente
CREATE OR REPLACE FUNCTION process_vente_s1()
RETURNS TRIGGER AS $$BEGIN
    IF NEW.ville = 'Casablanca' THEN
        RETURN NEW;
    ELSIF NEW.ville = 'Rabat' THEN
        INSERT INTO rabat_schema.Vente_S2 VALUES (NEW.*);
    END IF;
    RETURN NULL;
END;$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER trigger_vente_s1
BEFORE INSERT ON Vente_S1
FOR EACH ROW EXECUTE FUNCTION process_vente_s1();
```

3.2. Serveur Rabat

```sql
DROP TRIGGER IF EXISTS trigger_medecin_s2 ON Medecin_S2;
DROP TRIGGER IF EXISTS trigger_medicament_s2 ON s3_schema."Medicament";
DROP TRIGGER IF EXISTS trigger_stock_s2 ON Stock_S2;
DROP TRIGGER IF EXISTS trigger_vente_s2 ON Vente_S2;

-- Medecin
CREATE OR REPLACE FUNCTION process_medecin_s2()
RETURNS TRIGGER AS $$BEGIN
    IF NEW.ville = 'Rabat' THEN
        RETURN NEW;
    ELSIF NEW.ville = 'Casablanca' THEN
        INSERT INTO casa_schema.Medecin_S1 VALUES (NEW.*);
    END IF;
    RETURN NULL;
END;$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_medecin_s2
BEFORE INSERT ON Medecin_S2
FOR EACH ROW EXECUTE FUNCTION process_medecin_s2();

-- Medicament
CREATE OR REPLACE FUNCTION process_medicament_s2()
RETURNS TRIGGER AS $$BEGIN
    INSERT INTO Medicament_Prix_S2 ("idMedicament", "prixUnitaire")
    VALUES (NEW."idMedicament", NEW."prixUnitaire");
    RETURN NULL;
END;$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_medicament_s2
BEFORE INSERT ON s3_schema."Medicament"
FOR EACH ROW EXECUTE FUNCTION process_medicament_s2();

-- Stock
CREATE OR REPLACE FUNCTION process_stock_s2()
RETURNS TRIGGER AS $$BEGIN
    IF NEW.ville = 'Rabat' THEN
        RETURN NEW;
    ELSIF NEW.ville = 'Casablanca' THEN
        INSERT INTO casa_schema.Stock_S1 VALUES (NEW.*);
    END IF;
    RETURN NULL;
END;$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_stock_s2
```

```sql
BEFORE INSERT ON Stock_S2
FOR EACH ROW EXECUTE FUNCTION process_stock_s2();

-- Vente
CREATE OR REPLACE FUNCTION process_vente_s2()
RETURNS TRIGGER AS $$BEGIN
    IF NEW.ville = 'Rabat' THEN
        RETURN NEW;
    ELSIF NEW.ville = 'Casablanca' THEN
        INSERT INTO casa_schema.Vente_S1 VALUES (NEW.*);
    END IF;
    RETURN NULL;
END;$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_vente_s2
BEFORE INSERT ON Vente_S2
FOR EACH ROW EXECUTE FUNCTION process_vente_s2();
```

E. Jeux de données

```sql
-- #### Données globales ####

-- 1. Médicaments (10 produits)
INSERT INTO "Medicament" ("idMedicament", "nomMedicament", "forme",
"dosage", "prixUnitaire", "fabricant", "villeProduction") VALUES
('a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11', 'Doliprane', 'Comprimé',
'1000mg', 15.50, 'Sanofi', 'Casablanca'),
('a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a12', 'Amoxil', 'Gélule', '500mg',
45.00, 'GSK', 'Casablanca'),
('a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a13', 'Spasfon', 'Comprimé', '80mg',
28.00, 'Teva', 'Rabat'),
('a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a14', 'Augmentin', 'Sachet', '1g',
85.00, 'Glaxo', 'Casablanca'),
('a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a15', 'Ventoline', 'Inhalateur',
'100mcg', 55.00, 'GSK', 'Rabat'),
('a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a16', 'Voltarene', 'Gel', '1%', 35.00,
'Novartis', 'Casablanca'),
('a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a17', 'Gaviscon', 'Sirop', '250ml',
42.00, 'Reckitt', 'Rabat'),
('a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a18', 'Zyrtec', 'Comprimé', '10mg',
60.00, 'UCB', 'Casablanca'),
('a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a19', 'Aerius', 'Comprimé', '5mg',
72.00, 'MSD', 'Rabat'),
('a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a20', 'Difal', 'Comprimé', '50mg',
22.00, 'Laprophan', 'Casablanca');

-- 2. Stocks (2 par médicament : 1 Casa, 1 Rabat)
INSERT INTO "Stock" ("idStock", "idMedicament", "ville",
"quantiteDisponible", "seuilAlerte")
SELECT gen_random_uuid(), "idMedicament", 'Casablanca', 100, 10 FROM
"Medicament";

INSERT INTO "Stock" ("idStock", "idMedicament", "ville",
"quantiteDisponible", "seuilAlerte")
SELECT gen_random_uuid(), "idMedicament", 'Rabat', 80, 5 FROM
"Medicament";

-- 2.1 Insertation d'un stock < au seuil pour un médicament
INSERT INTO "Stock" ("idStock", "idMedicament", "ville",
"quantiteDisponible", "seuilAlerte")
```

```sql
SELECT gen_random_uuid(), "idMedicament", 'Rabat', 4, 5 FROM "Medicament"
WHERE "idMedicament" = 'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a20';

-- #### Données Casablanca ####

-- 1. Patients (6)
INSERT INTO "Patient" ("idPatient", "nomPatient", "prenomPatient",
"dateNaissance", "sexe", "adresse", "ville", "telephone") VALUES
(gen_random_uuid(), 'Alami', 'Ahmed', '1985-05-12', 'M', 'Rue Zerktouni',
'Casablanca', '0661112233'),
(gen_random_uuid(), 'Bennani', 'Sara', '1990-08-20', 'F', 'Bd Anfa',
'Casablanca', '0661445566'),
(gen_random_uuid(), 'Chraibi', 'Omar', '1975-03-15', 'M', 'Maarif',
'Casablanca', '0661778899'),
(gen_random_uuid(), 'Daoudi', 'Laila', '2000-12-01', 'F', 'Oasis',
'Casablanca', '0662112233'),
(gen_random_uuid(), 'El Fassi', 'Yassine', '1982-07-10', 'M', 'Ain Diab',
'Casablanca', '0662445566'),
(gen_random_uuid(), 'Filali', 'Zineb', '1995-11-25', 'F', 'Bourgogne',
'Casablanca', '0662778899');

-- 2. Medecins (3)
INSERT INTO "Medecin" ("idMedecin", "nomMedecin", "specialite", "ville",
"telephone") VALUES
('c1eebc99-9c0b-4ef8-bb6d-6bb9bd380a01', 'Dr. Mansouri', 'Généraliste',
'Casablanca', '0522112233'),
('c1eebc99-9c0b-4ef8-bb6d-6bb9bd380a02', 'Dr. Tazi', 'Cardiologue',
'Casablanca', '0522445566'),
('c1eebc99-9c0b-4ef8-bb6d-6bb9bd380a03', 'Dr. Amrani', 'Pédiatre',
'Casablanca', '0522778899');

-- 3. Consultations (6)
INSERT INTO "Consultation" ("idConsultation", "dateConsultation",
"diagnostic", "idPatient", "idMedecin")
SELECT gen_random_uuid(), CURRENT_DATE, 'Suivi Casa', "idPatient",
'c1eebc99-9c0b-4ef8-bb6d-6bb9bd380a01'
FROM "Patient" WHERE "ville" = 'Casablanca';

-- 4. Prescription (6)
INSERT INTO "Prescription" ("idPrescription", "datePrescription",
"idConsultation")
SELECT gen_random_uuid(), CURRENT_DATE, "idConsultation"
```

```sql
FROM "Consultation" WHERE "idMedecin" IN
('c1eebc99-9c0b-4ef8-bb6d-6bb9bd380a01','c1eebc99-9c0b-4ef8-bb6d-6bb9bd380
a02','c1eebc99-9c0b-4ef8-bb6d-6bb9bd380a03');

-- 5. Ventes (8 ventes)
INSERT INTO "Vente" ("idVente", "idPatient", "dateVente", "ville",
"montantTotal")
SELECT gen_random_uuid(), "idPatient", CURRENT_DATE, 'Casablanca', 0.00
FROM "Patient" WHERE "ville" = 'Casablanca' LIMIT 8;


-- Ligne Prescription
INSERT INTO "LignePrescription" ("idPrescription", "idMedicament",
"quantite", "posologie")
SELECT "idPrescription", 'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11', 1.00, '1
comprimé 3 fois par jour'
FROM "Prescription";

INSERT INTO "LignePrescription" ("idPrescription", "idMedicament",
"quantite", "posologie")
SELECT "idPrescription", 'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a12', 2.00, '1
gélule matin et soir'
FROM "Prescription"
LIMIT 3;

-- Ligne Vente
INSERT INTO "LigneVente" ("idVente", "idMedicament", "quantite",
"prixVente")
SELECT "idVente", 'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11', 1, 15.50
FROM "Vente";

INSERT INTO "LigneVente" ("idVente", "idMedicament", "quantite",
"prixVente")
SELECT "idVente", 'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a14', 1, 85.00
FROM "Vente"
LIMIT 4;

-- #### Donées Rabat ####

-- 1. Patients (6)
INSERT INTO "Patient" ("idPatient", "nomPatient", "prenomPatient",
"dateNaissance", "sexe", "adresse", "ville", "telephone") VALUES
(gen_random_uuid(), 'Gharbi', 'Mehdi', '1988-06-30', 'M', 'Agdal',
'Rabat', '0663112233'),
```

```sql
(gen_random_uuid(), 'Haddi', 'Meryem', '1992-09-14', 'F', 'Hay Riad',
'Rabat', '0663445566'),
(gen_random_uuid(), 'Idrissi', 'Anas', '1980-01-05', 'M', 'Souissi',
'Rabat', '0663778899'),
(gen_random_uuid(), 'Jebli', 'Salma', '2002-04-22', 'F', 'Hassan',
'Rabat', '0664112233'),
(gen_random_uuid(), 'Kabbaj', 'Karim', '1970-10-18', 'M', 'Les Orangers',
'Rabat', '0664445566'),
(gen_random_uuid(), 'Lahlou', 'Sofia', '1998-02-14', 'F', 'Ocean',
'Rabat', '0664778899');

-- 2. MEDECINS (3)
INSERT INTO "Medecin" ("idMedecin", "nomMedecin", "specialite", "ville",
"telephone") VALUES
('c2eebc99-9c0b-4ef8-bb6d-6bb9bd380b01', 'Dr. Benjelloun', 'Généraliste',
'Rabat', '0537112233'),
('c2eebc99-9c0b-4ef8-bb6d-6bb9bd380b02', 'Dr. Iraqi', 'Dermatologue',
'Rabat', '0537445566'),
('c2eebc99-9c0b-4ef8-bb6d-6bb9bd380b03', 'Dr. Jouahri', 'Ophtalmologue',
'Rabat', '0537778899');

-- 3. Consultations (6)
INSERT INTO "Consultation" ("idConsultation", "dateConsultation",
"diagnostic", "idPatient", "idMedecin")
SELECT gen_random_uuid(), CURRENT_DATE, 'Suivi Rabat', "idPatient",
'c2eebc99-9c0b-4ef8-bb6d-6bb9bd380b01'
FROM "Patient" WHERE "ville" = 'Rabat';

-- 4. Prescriptions (6)
INSERT INTO "Prescription" ("idPrescription", "datePrescription",
"idConsultation")
SELECT gen_random_uuid(), CURRENT_DATE, "idConsultation"
FROM "Consultation" WHERE "idMedecin" IN
('c2eebc99-9c0b-4ef8-bb6d-6bb9bd380b01','c2eebc99-9c0b-4ef8-bb6d-6bb9bd380
b02','c2eebc99-9c0b-4ef8-bb6d-6bb9bd380b03');

-- 5. Ventes (7 ventes)
INSERT INTO "Vente" ("idVente", "idPatient", "dateVente", "ville",
"montantTotal")
SELECT gen_random_uuid(), "idPatient", CURRENT_DATE, 'Rabat', 0.00
FROM "Patient" WHERE "ville" = 'Rabat' LIMIT 7;

-- Lignes Prescription
```

```sql
INSERT INTO "LignePrescription" ("idPrescription", "idMedicament",
"quantite", "posologie")
SELECT "idPrescription", 'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a13', 1.00, '1
comprimé si douleur'
FROM "Prescription";

INSERT INTO "LignePrescription" ("idPrescription", "idMedicament",
"quantite", "posologie")
SELECT "idPrescription", 'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a15', 1.00, '2
bouffées en cas de crise'
FROM "Prescription"
LIMIT 3;

-- Lignes Vente
INSERT INTO "LigneVente" ("idVente", "idMedicament", "quantite",
"prixVente")
SELECT "idVente", 'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a17', 1, 42.00
FROM "Vente";

INSERT INTO "LigneVente" ("idVente", "idMedicament", "quantite",
"prixVente")
SELECT "idVente", 'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a19', 1, 72.00
FROM "Vente"
LIMIT 3;
```

F. Requêtes réparties

Dans cette partie, les requêtes sont rédigées sur des **vues globales** reconstituées depuis les fragments des sites S1 (Casablanca) et S2 (Rabat). Cela permet d'interroger la base répartie comme une base unique, tout en conservant la répartition physique des données.

On suppose les vues globales suivantes :

1. Création des Vues gloables

1.1. Vue Patient_Global

```sql
CREATE OR REPLACE VIEW Patient_Global AS
SELECT
    i."idPatient",
    i."nomPatient",
    i."prenomPatient",
    i."dateNaissance",
    i."sexe",
    c."telephone",
    c."adresse",
    c."ville"
FROM Patient_Info_S1 i
JOIN Patient_Contact_S1 c
    ON i."idPatient" = c."idPatient"

UNION ALL

SELECT
    i."idPatient",
    i."nomPatient",
    i."prenomPatient",
    i."dateNaissance",
    i."sexe",
    c."telephone",
    c."adresse",
    c."ville"
FROM rabat_schema.Patient_Info_S2 i
JOIN rabat_schema.Patient_Contact_S2 c
    ON i."idPatient" = c."idPatient";
```

1.2. Vue Medecin_Global

```sql
CREATE OR REPLACE VIEW Medecin_Global AS
SELECT
    "idMedecin",
    "nomMedecin",
    "specialite",
    "ville",
    "telephone"
FROM Medecin_S1

UNION ALL

SELECT
    "idMedecin",
    "nomMedecin",
    "specialite",
    "ville",
    "telephone"
FROM rabat_schema.Medecin_S2;
```

1.3. Vue Consultation_Global

```sql
CREATE OR REPLACE VIEW Consultation_Global AS
SELECT
    "idConsultation",
    "dateConsultation",
    "diagnostic",
    "idPatient",
    "idMedecin"
FROM Consultation_S1

UNION ALL

SELECT
    "idConsultation",
    "dateConsultation",
    "diagnostic",
    "idPatient",
```

```sql
    "idMedecin"
FROM rabat_schema.Consultation_S2;
```

1.4. Vue Prescription_Global

```sql
CREATE OR REPLACE VIEW Prescription_Global AS
SELECT
    "idPrescription",
    "datePrescription",
    "idConsultation"
FROM Prescription_S1

UNION ALL

SELECT
    "idPrescription",
    "datePrescription",
    "idConsultation"
FROM rabat_schema.Prescription_S2;
```

1.5. Vue LignePrescription_Global

```sql
CREATE OR REPLACE VIEW LignePrescription_Global AS
SELECT
    "idPrescription",
    "idMedicament",
    "quantite",
    "posologie"
FROM LignePrescription_S1

UNION ALL

SELECT
    "idPrescription",
    "idMedicament",
    "quantite",
    "posologie"
FROM rabat_schema.LignePrescription_S2;
```

1.6. Vue Stock_Global

```sql
CREATE OR REPLACE VIEW Stock_Global AS
SELECT
    "idStock",
    "idMedicament",
    "ville",
    "quantiteDisponible",
    "seuilAlerte"
FROM Stock_S1

UNION ALL

SELECT
    "idStock",
    "idMedicament",
    "ville",
    "quantiteDisponible",
    "seuilAlerte"
FROM rabat_schema.Stock_S2;
```

1.7. Vue Vente_Global

```sql
CREATE OR REPLACE VIEW Vente_Global AS
SELECT
    "idVente",
    "idPatient",
    "dateVente",
    "ville",
    "montantTotal"
FROM Vente_S1

UNION ALL

SELECT
    "idVente",
    "idPatient",
    "dateVente",
    "ville",
```

```
    "montantTotal"
FROM rabat_schema.Vente_S2;
```

1.8. Vue LigneVente_Global

```SQL
CREATE OR REPLACE VIEW LigneVente_Global AS
SELECT
    "idVente",
    "idMedicament",
    "quantite",
    "prixVente"
FROM LigneVente_S1

UNION ALL

SELECT
    "idVente",
    "idMedicament",
    "quantite",
    "prixVente"
FROM rabat_schema.LigneVente_S2;
```

1.9. Vue Medicament_Global

```SQL
CREATE OR REPLACE VIEW Medicament_Global AS
SELECT
    "idMedicament",
    "nomMedicament",
    "forme",
    "dosage",
    "prixUnitaire",
    "fabricant",
    "villeProduction"
FROM s3_schema."Medicament";
```

2. Requêtes réparties sur les vues globales

2.1. Afficher les patients appartenant au site de Casablanca

Requête

```sql
SELECT *
FROM Patient_Global
WHERE "ville" = 'Casablanca';
```

Site d'exécution

Cette requête peut être exécutée sur **S1 (Casablanca)**, car les patients de Casablanca y sont stockés localement. Elle peut aussi être exécutée via la vue globale depuis S2.

Exemple de résultat

	idPatient uuid	nomPatient character varying (50)	prenomPatient character varying (50)	dateNaissance date	sexe character varying (20)	telephone character varying (20)	adresse character varying (50)	ville character varying (50)
1	86452271-07c9-4232-9e20-e31d51f8fe84	Alami	Ahmed	1985-05-12	M	0661112233	Rue Zerktouni	Casablanca
2	7ce85d0a-51f8-4f4e-a417-9d491621ebe4	Bennani	Sara	1990-08-20	F	0661445566	Bd Anfa	Casablanca
3	86a4b90c-ab1c-44b3-a923-8f6d967937aa	Chraibi	Omar	1975-03-15	M	0661778899	Maarif	Casablanca
4	1506d7b4-a57d-4112-bde0-4aba3a5b3311	Daoudi	Laila	2000-12-01	F	0662112233	Oasis	Casablanca
5	d6d78bfd-e0ba-43e9-9e0a-124d4712abd8	El Fassi	Yassine	1982-07-10	M	0662445566	Ain Diab	Casablanca
6	d87e8d75-80d0-4ecf-96d9-bea68edc96b1	Filali	Zineb	1995-11-25	F	0662778899	Bourgogne	Casablanca
7	550e8400-e29b-41d4-a716-446655440002	Alami	Sara	1990-05-15	F	0661223344	Rue Agdal	Casablanca
8	550e8400-e29b-41d4-a716-446655440002	Alami	Sara	1990-05-15	F	0661223344	Rue Agdal	Casablanca
9	550e8400-e29b-41d4-a716-446655440002	Alami	Sara	1990-05-15	F	0661223344	Rue Agdal	Casablanca
10	550e8400-e29b-41d4-a716-446655440002	Alami	Sara	1990-05-15	F	0661223344	Rue Agdal	Casablanca

Schéma 2 - Exemple résultat pour la requête n°1

2.2. Afficher les médicaments dont le prix unitaire est maximal

Requête

```sql
SELECT *
FROM Medicament_Global
WHERE "prixUnitaire" = (
SELECT MAX("prixUnitaire")
FROM Medicament_Global
```

```
);
```

Site d'exécution

Cette requête peut être exécutée sur **S1 ou S2**, car la table Medicament est répliquée et globalement accessible sur les deux sites. Le plus logique est de l'exécuter depuis **S1**.

Exemple de résultat

	idMedicament uuid	nomMedicament character varying (20)	forme character varying (20)	dosage character varying (20)	prixUnitaire numeric (8,2)	fabricant character varying (20)	villeProduction character varying (50)
1	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a...	Augmentin	Sachet	1g	85.00	Glaxo	Casablanca

Schéma 3 - Exemple résultat pour la requête n°2

2.3. Afficher les consultations d'un patient donné avec le nom du médecin

Requête

```sql
SELECT
    c."idConsultation",
    c."dateConsultation",
    c."diagnostic",
    m."nomMedecin"
FROM Consultation_Global c
JOIN Medecin_Global m
    ON c."idMedecin" = m."idMedecin"
WHERE c."idPatient" = 'ID_PATIENT_A_REMPLACER';
```

Site d'exécution

Cette requête est une **requête répartie**.
Elle peut être exécutée sur **S1** via les vues globales, avec accès éventuel aux données de S2.

Exemple de résultat

	idConsultation uuid	dateConsultation date	diagnostic text	nomMedecin character varying (50)
1	16f53615-9264-4615-ad77-1c59073513...	2026-04-02	Suivi Casa	Dr. Mansouri

Schéma 4 - Exemple résultat pour la requête n°3

2.4. Afficher tous les médicaments prescrits lors d'une consultation donnée

Requête

```sql
SELECT
    med."idMedicament",
    med."nomMedicament",
    lp."quantite",
    lp."posologie"
FROM Prescription_Global p
JOIN LignePrescription_Global lp
    ON p."idPrescription" = lp."idPrescription"
JOIN Medicament_Global med
    ON lp."idMedicament" = med."idMedicament"
WHERE p."idConsultation" = 'ID_CONSULTATION_A_REMPLACER';
```

Site d'exécution

Cette requête peut être exécutée sur **S1** ou **S2** via les vues globales.
Elle devient répartie si la consultation concernée se trouve sur l'autre site.

Exemple de résultat

	idMedicament uuid	nomMedicament character varying (20)	quantite numeric (8,2)	posologie character varying (255)
1	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a...	Doliprane	1.00	1 comprimé 3 fois par j...
2	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a...	Amoxil	2.00	1 gélule matin et soir
3	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a...	Spasfon	1.00	1 comprimé si douleur
4	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a...	Ventoline	1.00	2 bouffées en cas de cri...

Schéma 5 - Exemple résultat pour la requête n°4

2.5. Afficher le chiffre d'affaires total par ville

Requête

```sql
SELECT
    "ville",
    SUM("montantTotal") AS chiffre_affaires_total
FROM Vente_Global
GROUP BY "ville";
```

Site d'exécution

Cette requête est répartie, car elle agrège les ventes de Casablanca et de Rabat.
Elle peut être exécutée sur **S1**, qui interroge aussi les données distantes de **S2**.

2.6. Afficher le médicament le plus vendu sur l'ensemble du réseau

Requête

```sql
SELECT
    med."idMedicament",
```

```sql
    med."nomMedicament",
    SUM(lv."quantite") AS quantite_totale_vendue
FROM LigneVente_Global lv
JOIN Medicament_Global med
    ON lv."idMedicament" = med."idMedicament"
GROUP BY med."idMedicament", med."nomMedicament"
ORDER BY quantite_totale_vendue DESC
LIMIT 1;
```

Site d'exécution

Cette requête est une **requête globale répartie**, car elle exploite les lignes de vente des deux sites. Exécution conseillée sur **S1**.

Exemple de résultat

	idMedicament uuid	nomMedicament character varying (20)	quantite_totale_vendue bigint
1	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a...	Doliprane	6

Schéma 6 - Exemple résultat pour la requête n°5

2.7. Afficher les médicaments dont le stock est inférieur au seuil d'alerte

Requête

```sql
SELECT
    s."idStock",
    s."idMedicament",
    med."nomMedicament",
    s."ville",
    s."quantiteDisponible",
    s."seuilAlerte"
FROM Stock_Global s
```

```sql
JOIN Medicament_Global med
    ON s."idMedicament" = med."idMedicament"
WHERE s."quantiteDisponible" < s."seuilAlerte";
```

Site d'exécution

Cette requête peut être exécutée sur **S1** via la vue globale.
Elle devient répartie si l'on veut surveiller les deux villes en une seule fois.

Exemple de résultat

	idStock uuid	idMedicament uuid	nomMedicament character varying (20)	ville character varying (50)	quantiteDisponible smallint	seuilAlerte double precision
1	e770d9fb-ee7c-4298-a238-a2fe59b1ee...	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a...	Difal	Rabat	4	5

Schéma 8 - Exemple résultat pour la requête n°7

2.8. Afficher, pour chaque médecin, le nombre total de consultations réalisées

Requête

```sql
SELECT
    m."idMedecin",
    m."nomMedecin",
    COUNT(c."idConsultation") AS nombre_consultations
FROM Medecin_Global m
LEFT JOIN Consultation_Global c
    ON m."idMedecin" = c."idMedecin"
GROUP BY m."idMedecin", m."nomMedecin"
ORDER BY nombre_consultations DESC;
```

Site d'exécution

Cette requête est répartie, car elle parcourt les médecins et consultations de tout le réseau.
Exécution conseillée sur **S1**.

Exemple de résultat

	idMedecin uuid	nomMedecin character varying (50)	nombre_consultations bigint
1	c1eebc99-9c0b-4ef8-bb6d-6bb9bd380a...	Dr. Mansouri	6
2	c2eebc99-9c0b-4ef8-bb6d-6bb9bd380b...	Dr. Benjelloun	6
3	c1eebc99-9c0b-4ef8-bb6d-6bb9bd380a...	Dr. Tazi	0
4	c1eebc99-9c0b-4ef8-bb6d-6bb9bd380a...	Dr. Amrani	0
5	c2eebc99-9c0b-4ef8-bb6d-6bb9bd380b...	Dr. Iraqi	0
6	c2eebc99-9c0b-4ef8-bb6d-6bb9bd380b...	Dr. Jouahri	0

Schéma 9 - Exemple résultat pour la requête n°8

2.9. Afficher les patients ayant effectué une consultation dans une ville et un achat dans l'autre

Requête

Comme la table **Consultation** ne contient pas directement **ville** dans le schéma global, on déduit la ville de consultation via le médecin.

```sql
SELECT DISTINCT
    p."idPatient",
    p."nomPatient",
    p."prenomPatient"
FROM Patient_Global p
JOIN Consultation_Global c
    ON p."idPatient" = c."idPatient"
JOIN Medecin_Global m
    ON c."idMedecin" = m."idMedecin"
JOIN Vente_Global v
    ON p."idPatient" = v."idPatient"
WHERE m."ville" <> v."ville";
```

Site d'exécution

Cette requête est clairement **répartie**, car elle compare consultation et achat entre les deux sites.
Elle doit être exécutée sur **S1** ou **S2** via les vues globales.

Exemple de résultat

idPatient uuid	nomPatient character varying (50)	prenomPatient character varying (50)

Schéma 10 - Exemple résultat pour la requête n°9

2.10. Afficher les patients ayant acheté au moins un médicament qui figurait dans leur prescription

Requête

```sql
SELECT DISTINCT
    p."idPatient",
    p."nomPatient",
    p."prenomPatient"
FROM Patient_Global p
JOIN Consultation_Global c
    ON p."idPatient" = c."idPatient"
JOIN Prescription_Global pr
    ON c."idConsultation" = pr."idConsultation"
JOIN LignePrescription_Global lp
    ON pr."idPrescription" = lp."idPrescription"
JOIN Vente_Global v
    ON p."idPatient" = v."idPatient"
JOIN LigneVente_Global lv
    ON v."idVente" = lv."idVente"
WHERE lp."idMedicament" = lv."idMedicament";
```

Site d'exécution

Cette requête est une **requête répartie complexe**, car elle mobilise les consultations, prescriptions, ventes et lignes de vente sur l'ensemble du réseau.
Exécution conseillée sur **S1** via les vues globales.

Exemple de résultat

	idPatient uuid	nomPatient character varying (50)	prenomPatient character varying (50)
1	86452271-07c9-4232-9e20-e31d51f8fe84	Alami	Ahmed
2	7ce85d0a-51f8-4f4e-a417-9d491621ebe4	Bennani	Sara
3	86a4b90c-ab1c-44b3-a923-8f6d967937aa	Chraibi	Omar
4	d87e8d75-80d0-4ecf-96d9-bea68edc96b1	Filali	Zineb
5	d6d78bfd-e0ba-43e9-9e0a-124d4712abd8	El Fassi	Yassine
6	1506d7b4-a57d-4112-bde0-4aba3a5b3311	Daoudi	Laila

Schéma 11 - Exemple résultat pour la requête n°10

G. Analyse critique

Notre architecture présentes beaucoup d'avantages mais aussi certaines limites :

Avantages :

- requêtes locales rapides
- bonne scalabilité
- isolation des sites
- maîtrise du flux réseau
- gestion des données incohérentes (données d'un site ne peut être insérées sur l'autre site)
- création de vues pour faciliter la visualisation des données et rendre transparent les fragmentations et réplications
- améliore la fiabilité aux serveurs de bases de données (si un serveur est down, certaines données peuvent quand même être accessibles)

Limites :

- complexité de mise en place
- gestion des transactions distribuées
- cohérence inter-site plus difficile.
- nécessité de gérer plusieurs serveurs de bases de données au lieu d'un seul
- maintenance plus compliquée

H. Conclusion

Ce projet, finalisant le module du cours de bases de données réparties, nous a permis de mettre en pratique de façon concrète la théorie vu pendant le cours. Nous avons mis en place une architecture multi-sites, et avons pu mieux comprendre les problématiques liées à la répartition des données, à la fois en termes de performance, de disponibilité ou d'organisation globale du système.

Le travail effectué sur la fragmentation, aussi bien horizontale que verticale (et des fois les deux sur une seule table), nous a obligés à réfléchir à la façon dont les données vont être utilisées en pratique. Cela nous a forcé à définir et puis construire une structure cohérente, qui correspond aux besoins définis dans l'énoncé tout en limitant les échanges entre les différents sites. La dernière partie sur la mise en place de vues globales a aussi été importante, car c'est elle qui permet d'avoir une utilisation simple de la base de données malgré sa distribution physique.

En revanche, ce projet démontre également que ce type d'architecture reste complexe à mettre en place. La gestion de la cohérence entre les sites, les transactions distribuées ou encore la configuration technique avec PostgreSQL demandent beaucoup de travail en amont et de rigueur.

Finalement, faire ce projet nous a permis d'obtenir une vision plus concrète des bases de données réparties, car il met bien en évidence les avantages de ce type de système, mais aussi les contraintes qu'il impose en termes de conception et de maintenance.